📘

📘 Project Tutorials — Step by Step (All Projects + Kit)

📘 Your complete, follow-along setup guide for all four products — the **GODSHAND film kit** plus the three software projects (**Convergence**, **SafeScreen**, **Solana Sniper**). Each section goes top to bottom, click by click, so you — or a buyer — never have to guess.

> **Note on the Google Drive link:** I couldn't open the Drive folder you shared — Drive folder links load their file list in the browser and can't be read from a share URL, and I don't have a Drive connector. This guide is built from your own project pages in this workspace instead, which are accurate and current. If you want any project's *exact* file names/folder layout baked in, paste that project's file list here (or drop the project's `.zip` into our chat) and I'll fold it in.

How to use this guide

- Each project is **self-contained** — jump straight to the one you need.
- **What you need** lists prerequisites — get those first.
- Steps are in order. Don't skip; each unlocks the next.
- **Honest limits** is what the product does *not* do. Keep it in every listing — it's your refund shield.

---

1) 🎬 GODSHAND — Cinematic AI Film Kit

Open the listing + setup sheet

What it is: turns your character image into graded, HD, motion-consistent film shots — running on a free Kaggle GPU. This is the exact pipeline we set up

together for the 2-minute cut of *The Saga of the Beholder*.

What you need

- [ ] A free **Kaggle** account, phone-verified (unlocks GPUs)
- [ ] A free **HuggingFace** account + a read **access token** (optional, but gives faster model downloads)
- [ ] The **GODSHAND kit** uploaded as a Kaggle **Dataset** — it must contain a `src/` folder with `render_motion.py` inside
- [ ] Your **character/anchor images** as a Kaggle Dataset (or use the samples)

Step by step

1. **Upload the kit as a Kaggle Dataset.** Take the kit ZIP (`GODSHAND_KIT_AUTOPILOT.zip`) → Kaggle → *Datasets* → *New Dataset* → upload. Kaggle auto-extracts it, so the `src/` folder is detected automatically.
2. **Create/import the notebook.** New Notebook → *File* → *Import Notebook* → upload `rainman-wan-multichar.ipynb` (the version with the 2-minute cut).
3. **Attach your datasets.** In the notebook's right sidebar: **Add Input** → **Datasets** → add (a) your GODSHAND kit dataset and (b) your `saga-custom-characters` refs dataset.
4. **(Optional) Add your HF token.** *Add-ons* → *Secrets* → new secret named `HF_TOKEN` → paste your token. Skipping this still works; downloads are just slower.
5. **Set the machine.** Right sidebar: **Accelerator = GPU T4 × 2**, **Internet = On**.
6. **Run All.**
7. **Confirm the setup log.** You should see `GODSHAND kit mounted…` and then `AUTO-DERIVED from story: total=28 shots` — that's the 2-minute cut building correctly.
8. **Let it render.** It renders shot by shot and skips any already finished.

> ⚠️ **It runs on ONE T4, even on the "T4 × 2" machine.** A built-in RAM guard only ever launches one render server (two would exceed the 32 GB host

RAM and crash). So shots render **one at a time**, not in parallel — this is expected, not a bug.

The 2-minute cut (already configured for you)

- **28 shots, ~1:57** — the five opening scenes (`i01_jungle → i05_between`).
- `rejection_samples = 1` — one take per shot (fastest).
- **Net render time: ~4.5–5.5 hours** on a single T4 (~8–9 min/shot).

If a session stops before finishing (resume)

With the free **~2 GPU-hours/week**, it won't finish in one go — it stops cleanly at the budget and saves `Beholder_progress.zip` + a `STATUS.txt` listing what's left.

1. Next session: **Add Input** → **Your Work** → **this notebook** → **its Output**.
2. **Run All** again — it detects finished clips and picks up where it stopped.
3. When all 28 are done, it auto-stitches the final master into the zip.

Honest limits

- Guided notebook, **not one-click**; needs a Kaggle account (+ optional HF token).
- Single-T4 sequential rendering; ~8–9 min/shot.
- Stylized AI output — occasional artifacts (hands, tiny details).
- Check each underlying model's **license** before selling what you render.

---

2) 🎯 Convergence Framework — ₹499

Open the listing

What it is: a thinking framework + ready-made LLM prompts to define the *right* objective before you build — so you can decide in an hour whether a goal is even formalizable.

What you need

- The kit ZIP (a PDF + a few markdown files). **No install, no GPU.**
- Any LLM (ChatGPT / Claude / Gemini).

How to use it

1. Open `Convergence_Under_Constraint.pdf` — read the core method first.
2. Follow `README_objective_function.md` — walk Steps 0–8 to write your objective function.
3. Paste `SystemPrompt_Code_Convergence_Deterministic.md` into your LLM as a *system prompt* before coding.
4. Use `AI_Code_Review_Prompt_v2.md` to review code against the objective (not just syntax).
5. Study `Trading_Bot_Convergence_Framework_v3.md` as a fully worked example, then copy the pattern for your own system.

To make it demo-ready (before selling)

- Apply it to **3–4 real goals**, including one deliberately messy / non-formalizable one, and show it correctly flags the messy one.
- Record a short worked example: *real goal → run the framework → verdict.*

Honest limits

- It's a **thinking discipline, not runnable code** — the value is the judgment, not an executable.

---

3) 🛡️ SafeScreen — ₹1,999

Open the listing

What it is: a trained on-device content-moderation model + inference SDK — flags/masks violence, gore, and NSFW in images, with an OCR layer for text. Runs offline, no cloud.

> ⚠️ **Redact secrets FIRST.** `nsfw_fix.py` contains a hardcoded Kaggle username + API key. Delete those lines and **rotate the key** before you

zip or share anything.

What you need

- Python + a fresh virtual environment. Runs on **CPU** (onnxruntime) — no GPU required.

How to run

1. Create a fresh venv and install the project's dependencies.
2. **Sanity check first:** confirm `toxic_detector.onnx` loads and classifies **one** image end-to-end with onnxruntime on CPU.
3. Run the inference scripts: `inference_onnx`, `inference_tflite`, or `batch_inference`.
4. Build a **held-out test set you did NOT train on** → report **precision, recall, F1, a confusion matrix, and false-positive rate — per class**.
5. **Live demo:** a minimal Gradio or FastAPI page — upload an image, get the flagged label + confidence, fully offline. Screen-record it.

Honest limits

- Per-class accuracy varies — only market the classes where recall is genuinely strong; flag the weak ones. Buyers must test on their own data.

---

4) 📈 Solana Sniper — ₹2,499

Open the listing

What it is: the full source + a rigorous **6-month paper-trading study** of a Solana memecoin sniper (ingestion, ML signal layer, risk manager, React/TS dashboard, Monte Carlo risk analysis). Sold as a **study**, not a live money-maker.

> ⚠️ **Redact secrets FIRST.** Delete `.env`, `.env1`, and `gmgn_keys/gmgn_private.pem`; **rotate** the wallet + API keys (assume burned); add a `.gitignore` and a clean `.env.example`.

How to run (paper mode only)

1. **Hard safety rule:** never connect real funds, real keys, or mainnet execution. Paper / backtest only.
2. Set up a fresh environment and install dependencies.
3. Re-run the **Monte Carlo / backtest harness** on historical data.
4. Report the **equity curve, win rate, and max drawdown — losing runs included**, honestly.
5. Reproduce the study's output from the shipped ZIP with a **single command**, and record it.

Honest limits

- Paper-tested only; live performance unverified. **Not financial advice; past results don't predict future results.**

🧰 Handy notes for all four

- **Free tool to run/fix code:** Antigravity (free agentic IDE) for the CPU projects; fall back to Cursor (free) or aider if it throttles. GPU work (the film) stays on Kaggle.
- **Always strip secrets** (`.env`, `.pem`, keys) and add a `.env.example` before zipping any project.
- **Record proof on a clean machine/account** so a buyer's first run matches your demo.
- **Fastest first rupee:** publish **Convergence (₹499)** first — it's clean, needs zero redaction, and is the shortest path to a sale.